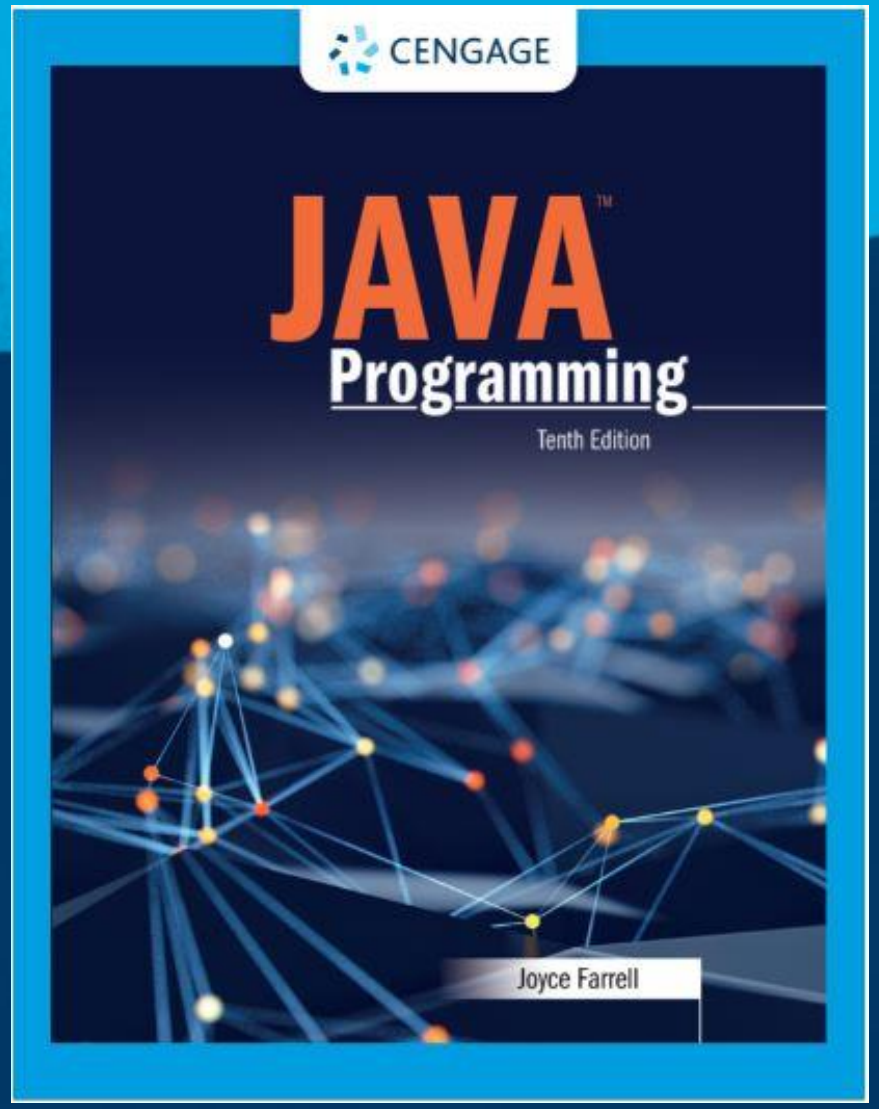


Java Programming, 11e

Chapter 13: Collections and Generics

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Chapter Objectives

By the end of this chapter, you should be able to:

13.1 Use the `Collection` interface

13.2 Use the `List` interface

13.3 Use the `ArrayList` class

13.4 Use the `LinkedList` class

13.5 Use iterators

13.6 Create generic classes

13.7 Create generic methods

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.1 Understanding the `Collection` Interface (1 of 2)

Collection

- A group of objects that can be operated on together
- Like arrays, hold references to objects and can be used as a group
- Unlike arrays:
  - Arrays need to be assigned a size or capacity that cannot change
  - Collections are dynamically resizable
  - Arrays can hold primitive data types such as `int` and `double`; collections cannot
  - Collections can hold objects that are members of classes

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.1 Understanding the `Collection` Interface (2 of 2)

Java `Collections` framework

- Set of interfaces and classes
- Help manage groups of objects
- Define collections and the classes that define them
- Almost all collections in Java are derived from the `Collection` interface

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 13.1: Knowledge Check

A(n) _____ describes a group of objects that can be operated on together.

a. set

b. instance

c. collection

d. group

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 13.1: Knowledge Check Answer

A(n) _____ describes a group of objects that can be operated on together.

Answer: c. A collection describes a group of objects that can be operated on together.

To be considered a collection, the group size must be able to increase and decrease.

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.2 Understanding the `List` Interface (1 of 1)

List

- A collection of an ordered sequence of objects

`List` interface

- Extends `Collection`
- Elements can be inserted or accessed by their position in the list
- It can contain duplicate elements
- Contains all `Collection` methods but defines its own

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.3 Using the `ArrayList` Class (1 of 4)

`ArrayList`

- Concrete class
- Extends `Collection` abstract classes
- Descends from `AbstractList`
- Used to create containers that store lists of objects

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.3 Using the `ArrayList` Class (2 of 4)

Advantage of `ArrayList` over `Arrays`

- `ArrayList` is dynamically resizable
  - You can add or remove an item at any point and the array size changes to accommodate

Capacity of an `ArrayList`

- Number of items it can hold without having to increase its size
- Capacity is greater than or equal to its size
- You can specify a capacity

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.3 Using the `ArrayList` Class (3 of 4)

```
import java.util.ArrayList;
public class ArrayListDemo
{
   public static void main(String[] args)
   {
      ArrayList<String> names = new ArrayList<String>();
      names.add("Anee");
      display(names);
      names.add("Brian");
      display(names);
      names.add("Zelda");
      display(names);
      names.add(2, "Juan");
      display(names);
      names.remove(1);
      display(names);
      names.set(0, "Dinh");
      display(names);
      names.add("Lee");
      display(names);
   }
   public static void display(ArrayList<String> names)
   {
      System.out.println("\nThe size of the list is " + names.size());
      System.out.println(names);
   }
}
```

Figure 13-1: The `ArrayListDemo` application

```
The size of the list is 1
[Anee]

The size of the list is 2
[Anee, Brian]

The size of the list is 3
[Anee, Brian, Zelda]

The size of the list is 4
[Anee, Brian, Juan, Zelda]

The size of the list is 3
[Anee, Juan, Zelda]

The size of the list is 3
[Dinh, Juan, Zelda]

The size of the list is 4
[Dinh, Juan, Zelda, Lee]
```

Figure 13-2: Execution of the `ArrayListDemo` application

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.3 Using the `ArrayList` Class (4 of 4)

Using the `ArrayList` class

- Can hold any reference type, but not a primitive type
- Can use with objects instantiated from your own classes

Class natural ordering

- Sort, such as `String`s to alphabetical order
- `compareTo()` method
  - Must be named `compareTo()`, accept parameter that indicates comparison object type, return an `int`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 13.2: Discussion

1. Describe the `List` interface.
2. Why is it an advantage that `ArrayList` is dynamically resizable?

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 13.2: Discussion Debrief

1. The `List` interface extends `Collection`. Elements can be inserted or accessed by their position in the list. It can contain duplicate elements.
2. Its size can change during program execution, so that you can add or remove an item at any point, and the array size resizes automatically.

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.4 Using the `LinkedList` Class (1 of 4)

Linked list

- Collection of data elements
- Each element holds the memory address of one or more elements to which it is logically connected
- Single-linked list holds only the address of the item that follows it
- Double-linked list holds addresses for the preceding and following elements

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.4 Using the `LinkedList` Class (2 of 4)

`LinkedList` class

- Use an `import` statement
- `import java.util.LinkedList;`
- `Import java.util.*;`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.4 Using the `LinkedList` Class (3 of 4)

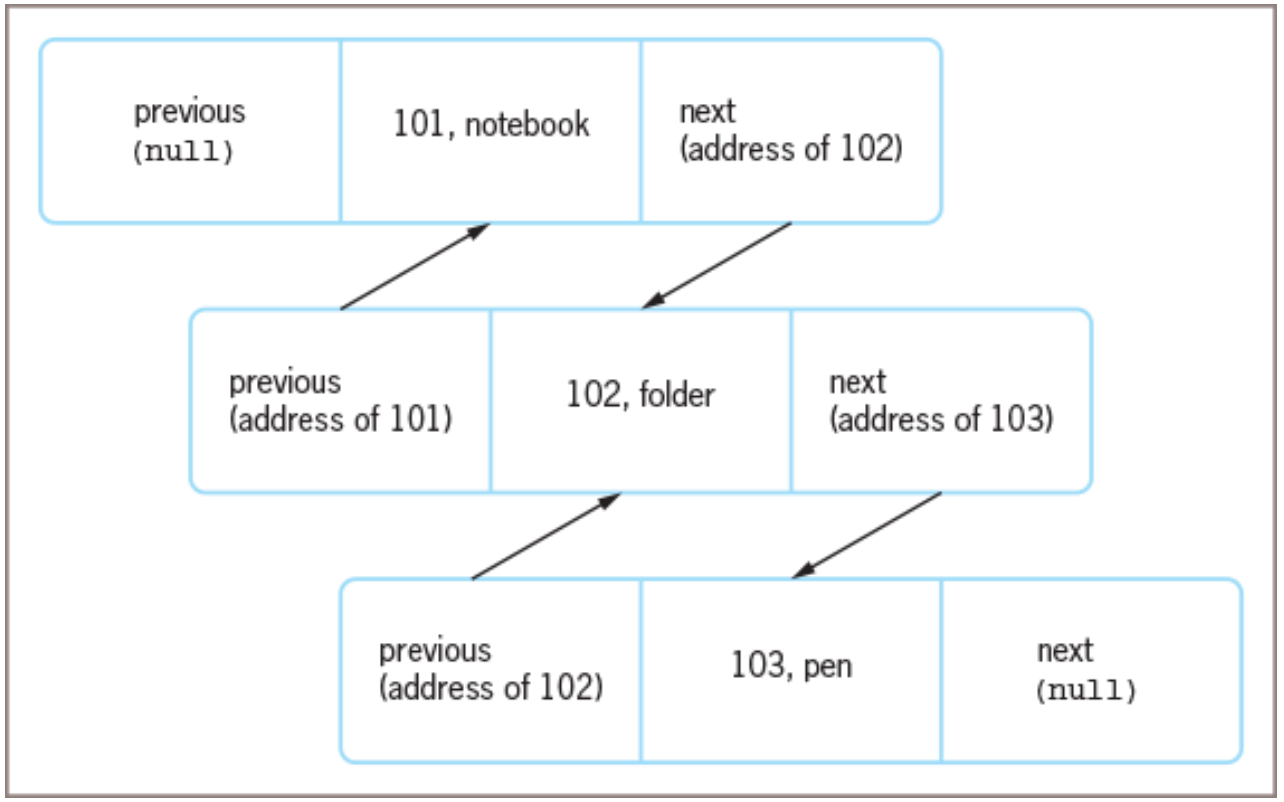


Figure 13-17: Three `SchoolSupply` objects stored in a linked list

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.4 Using the `LinkedList` Class (4 of 4)

Comparing `LinkedList` and `ArrayList`

- `LinkedList` works better when you must move or relocated a lot of the data stored there
- `ArrayList` performs less efficiently when you move data
- `LinkedList` works less efficiently when you have to sequentially store and retrieve a lot of data
- `ArrayList` works better when you have to sequentially store and retrieve a lot of data

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.5 Using Iterators (1 of 2)

Iterator

- Object that can be used to loop through the elements in a `Collection`
- You can use Java's behind-the-scenes iterator or create your own
  - `import java.util.Iterator;`
  - `Import java.util.*;`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.5 Using Iterators (2 of 2)

`Iterator` object has access to three methods:

- `hasNext()`
  - Returns a Boolean value indicating whether another object exists in a `Collection`
- `next()`
  - Returns the next object in a `Collection`
- `remove()`
  - Removes the last element returns by the `Iterator`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.6 Creating Generic Classes (1 of 3)

Generic programming

- Programming style
- Code written with data types to be specified later
- Enable nonprimitive data types to be parameters when defining classes, interfaces, and methods
- Used by developers to create `ArrayList`, `LinkedList`, and `Container` classes

Type parameter

- Identifier that specifies the name of the generic type
- Provide way to reuse programming logic for different types

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.6 Creating Generic Classes (2 of 3)

Generic class

- Uses a type parameter
- Looks like a nongeneric class declaration, but the class name is followed by a type parameter section
  - Composed of one or more type parameters
  - Between angle brackets < >
  - If more than one type parameter, separated by commas

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.6 Creating Generic Classes (3 of 3)

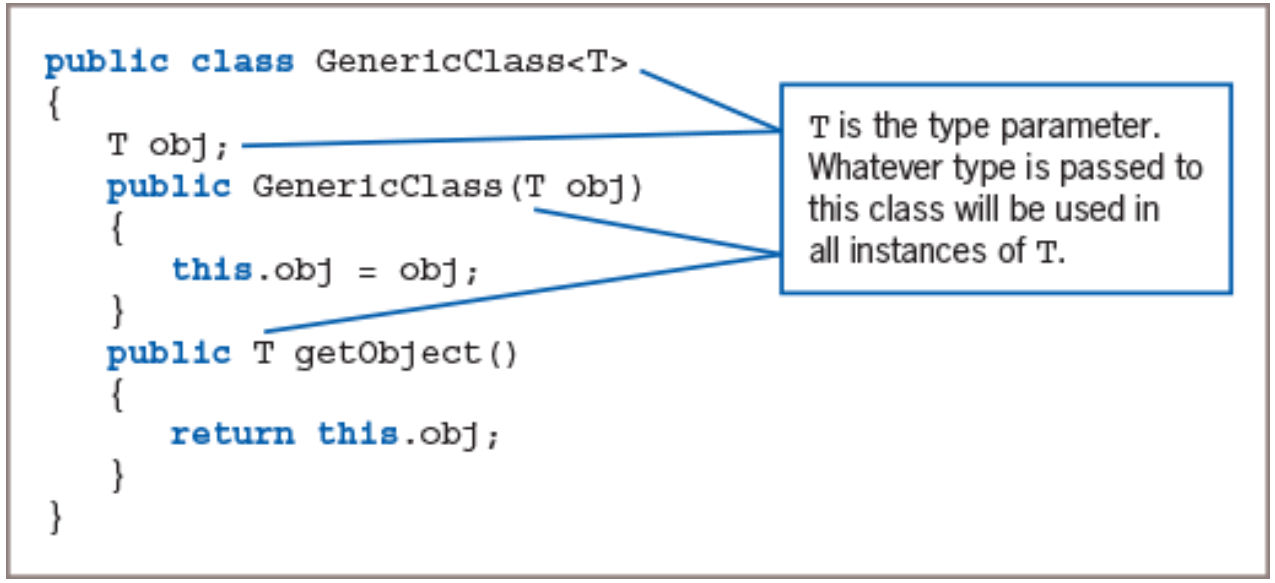


Figure 13-25: The `GenericClass` class

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.7 Creating Generic Methods (1 of 2)

Generic method

- Uses one or more parameters
- Type parameter must be a reference type
- Multiple type parameters

Creating generic methods with more than one type parameter

- Separated by commas
- Each needs its own identifier

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

13.7 Creating Generic Methods (2 of 2)

```
public class PreferredCustomer
{
   private int custNo;
   private String name;
   public PreferredCustomer(int custNo, String name)
   {
      this.custNo = custNo;
      this.name = name;
   }
   public String toString()
   {
      String string = "Preferred Customer #" + custNo + " " + name +
         "\n   Thank you for being a preferred customer";
      return string;
   }
}
```

Figure 13-32: The `PreferredCustomer` class

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 13.3: Case Study

You have been asked to create a program that uses the `LinkedList` class, iterators, and generic classes and methods.

- Give an example of why it is better to use a `LinkedList` for your program that has to remove an element.
- For what purpose would you use an iterator? What type of statement do you need to use to create an `Iterator` object?
- When you create a generic method, where you do place with type parameter?

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 13.3: Case Study Debrief

- When you remove an element from a `LinkedList`, you only change the references stored in the items that precede and follow it.
- You can use one of two `import` statements to create an `Iterator` object.
- Type parameters provide a way for you to reuse the same programming logic with different types.
- You place the type parameter in the method header within angle brackets before the method's return type.

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Don’t Do It

- Don’t use an array if the size of a collection must grow or shrink during execution
- Don’t use a `Collection` if you must hold elements that are primitive data types
- Don’t omit the data type between angle brackets in an `ArrayList` declaration
- Don’t choose an `ArrayList` over a `LinkedList` for storing a collection if performance is important and a lot of data must be moved in a program
- Don’t choose a `LinkedList` over an `ArrayList` if performance is important and a program must sequentially store and retrieve a lot of data
- Don’t try to use a primitive data type method that uses a generic parameter

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Self-Assessment

1. Are collections dynamically resizable? Explain.
2. List methods in the `List` interface.
3. Name two concrete classes that extend `Collection` abstract classes.
4. Which performs less efficiently when you move data: `ArrayList` or `LinkedList`?
5. Explain why it is more efficient to use an `Iterator` than a `for` loop to process in every element in a `Collection`.
6. Explain why developers use the generic class technique to create `Container` classes.
7. How do you separate multiple type parameters in a generic method?

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Summary

Click the link to review the objectives for this presentation.

Link to Objectives

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.